

Project 1 – Low-Overhead Timing-Slack-Aware “Warning” Flip-Flop

Project brief (for students)

Modern low-power and near-threshold designs suffer from timing errors due to voltage scaling, PVT variations, and aging. Classic solutions such as Razor flip-flops add significant area, power, and design complexity, while more recent timing-error-tolerant flip-flops still incur non-trivial overhead. In this project, you will propose a new **timing-slack-aware flip-flop (“warning FF”)** that can detect imminent timing violations with minimal extra transistors and simple clocking, suitable for integration into standard digital pipelines. Using a Synopsys RTL-to-GDSII flow (Design Compiler + ICC), you will: (1) design the warning FF and a reference D-FF in HDL, (2) integrate them into a small multi-stage pipeline, (3) perform synthesis, place & route, and timing/power analysis, and (4) quantitatively compare area, power, and error-detection capability under different clock periods and voltages (multi-corner if possible). Expected deliverables: Verilog/SystemVerilog code, synthesis and P&R reports, timing diagrams explaining the detection mechanism, and a 6–8 page technical report highlighting what is novel in your design versus prior timing-error-tolerant flip-flops.

Recent reference to study

- Razor concept (classical baseline): D. Ernst et al., “Razor: Circuit-level correction of timing errors,” *IEEE Micro*, 2004 (seminal paper, still the baseline).
- Example of recent timing-error-tolerant FF idea: “Timing Error Tolerant Circuit Technique for Low-Power Control Logic,” 2023 (conference article proposing a simple timing-error-tolerant method).

Suggested IEEE target journal

- **Primary target:** *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (TVLSI) – focuses on VLSI circuit and system-level innovations, including low-power and robust design.
- **Alternative:** *IEEE Transactions on Circuits and Systems I (TCAS-I)*, if your work is more circuit-theoretic.

Project 2 – Configurable-Accuracy Approximate MAC Unit for ML/DSP

Project brief (for students)

Approximate arithmetic is widely used to reduce energy and area in error-tolerant

applications such as image processing and machine learning, but many designs use fixed-accuracy multipliers or adders. More recent works propose **configurable accuracy** multipliers, yet there is still room for simpler hardware control, finer accuracy levels, or better VLSI PPA trade-offs. In this project, you will design a **16-bit (or 8-bit) multiply-accumulate (MAC) unit with run-time configurable accuracy**, integrating several approximate multiplier/adder variants under a unified control interface. Using Synopsys Design Compiler and ICC, you will: (1) implement at least two approximate multiplier architectures plus a precise baseline, (2) design a control scheme (registers/FSM) that selects accuracy mode at run time, (3) synthesize and place & route the MAC in different modes and extract area, delay, and power, and (4) evaluate numerical error on a small ML/DSP kernel (e.g., dot product or FIR filter) for different accuracy settings. Expected deliverables: HDL code, synthesis and layout reports, error–energy plots, and a 6–8 page report clearly explaining the novelty of your configurability mechanism compared to existing configurable multipliers.

Recent references to study

- J. Liang et al., “AMCAL: Approximate Multiplier with Configurable Accuracy Levels,” *IEEE* (2023) – configurable-accuracy multiplier concept.
- A. energy-efficient accuracy configurable multiplier: “An Energy-Efficient Generic Accuracy Configurable Multiplier Based on ...,” *IEEE* (2023).
- Broader approximate multiplier context: “Improving the accuracy of approximate multipliers based on the characteristics of 4:2 compressors,” 2023.; and older but important works like low-power approximate multipliers using encoded partial products.
- Design-space-exploration for approximate units: “FPAX: A Fast Prior Knowledge-Based Framework for DSE in Approximate Computing,” 2023.

Suggested IEEE target journal

- **Primary target:** *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* – strong fit for digital approximate arithmetic blocks implemented and evaluated at VLSI level.
 - **Alternative / more thematic:** *IEEE Journal on Emerging and Selected Topics in Circuits and Systems (JETCAS)* – especially for special issues on approximate/stochastic computing or energy-efficient ML hardware.
-

Project 3 – Adaptive Multi-Level Power-Gating Controller for a Small SoC Block

Project brief (for students)

Conventional power-gating in SoCs typically uses static policies (on/off) with coarse-grain sleep decisions, which often leaves leakage savings on the table or causes performance penalties. Recent research on adaptive power management combines workload monitoring, DVFS, and power gating, but implementations are typically complex multi-core systems. In this project, you will design a **lightweight adaptive power-gating controller** for a small single-core block (e.g., ALU or simple RISC pipeline) that supports multiple power states (ON / LIGHT-SLEEP / DEEP-SLEEP) and uses local activity history and simple prediction/hysteresis to decide transitions. Using Synopsys tools, you will: (1) model the functional block and controller in HDL, (2) define power domains and modes (using UPF/CPF if available), (3) synthesize and implement the design with Design Compiler and ICC including power-gating cells, and (4) evaluate leakage savings, wake-up latency, and performance impact on synthetic workloads. Expected deliverables: RTL, power-intent files (if used), synthesis/P&R and power analysis reports, state-machine diagrams for the controller, and a 6–8 page report that clearly states your novel policy/architecture compared to prior adaptive power-management techniques.

Recent references to study

- Example of modern adaptive power-management concept: “Adaptive Power Management Techniques in Multi Core VLSI Designs,” 2025 – combines workload monitoring with DVFS and core-level power gating.
- Survey/overview of low-power VLSI techniques (DVFS, power gating, clock gating, etc.) as background: “Design and Optimization of Low-Power VLSI Circuits for High-Performance Computing Systems,” 2025.

Suggested IEEE target journal

- **Primary target:** *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* – regularly publishes work on power-management techniques and low-power digital design.
- **Alternative:** *IEEE Transactions on Circuits and Systems I (TCAS-I)* if your emphasis is more on the control/algorithmic aspect of the power controller.

- **Common minimum for everyone :**
 - **Clean RTL (Verilog/SystemVerilog).**
 - **Synthesis in Synopsys Design Compiler with basic timing and area reports.**
 - **Simple ICC place & route for at least the core block and basic post-layout timing.**
 - **Short report (≈6 pages) and presentation summarizing novelty and PPA results.**
- **Other expectations for each group:**
 - **Do a short literature/patent review section (2–3 recent IEEE/elsevier/patent refs beyond the ones I gave).**
 - **Explore two design points (e.g., low-power vs. high-speed constraints) and compare PPA trade-offs.**
 - **Include a clear novelty statement (bullet list: what is new vs. [Razor / AMCAL / adaptive PM papers]) and a simple idea of how this could map to a patent claim (e.g., “a flip-flop comprising ...”).**
 - **For at least one project, push ICC to a more complete flow (clock tree, basic routing congestion analysis).**

How to form the three teams

- **Team 1 – Warning Flip-Flop: ideally 1 MS + 1 UG (MS leads literature/novelty, UG leads testbenches and scripting).**
- **Team 2 – Configurable Approximate MAC: good for students who like ML/DSP; mix levels the same way.**
- **Team 3 – Adaptive Power-Gating Controller: good for students interested in architecture/SoC and power-intent (UPF/CPF) basics.**

Required outcomes for a publishable-quality project

Each team:

1. Technical report (conference-style, 6–8 pages)

Structure it like an IEEE paper (two-column):

- Title, authors, abstract, keywords.
- Introduction: problem, motivation, contributions (2–4 bullet “This work proposes...”).
- Background & related work: summarize at least 3 recent works (2 IEEE + 1 other) and clearly state what is missing.
- Proposed method/architecture: diagrams, equations (if needed), timing/state diagrams, novelty clearly highlighted.
- Implementation details: technology node (generic), Synopsys flow (DC + ICC), constraints, libraries, testbench assumptions.
- Results: tables and plots for PPA (power, performance, area) and, where relevant, error metrics or energy savings.
- Discussion: comparison to prior work, limitations, future improvements.
- Conclusion and references (IEEE style).

2. Working RTL + scripts + documentation

- Clean, synthesizable Verilog/SystemVerilog for all blocks.
- Synopsys scripts (DC + ICC) to reproduce main results (synthesis constraints, P&R scripts).
- Short README explaining how to run simulations, synthesis, and P&R.

3. Experimental evidence suitable for a paper

At minimum:

- Baseline vs. proposed design comparison under at least **two operating corners or constraint sets** (e.g., low-power vs. high-speed).
- Quantitative PPA tables (area, Fmax or delay, dynamic and/or leakage power).

- For approximate MAC: accuracy/error vs. energy or area for each mode.
- For warning FF: percentage of timing violations detected vs. timing slack, with overhead numbers.
- For adaptive power-gating: leakage savings, wake-up latency, and throughput impact across several workloads.

4. Patent-oriented novelty statement (1 page)

- 3–5 bullet points: “What is new compared to prior art?”
- Block diagrams and a short text describing how the idea could be expressed as a method or circuit claim (“A flip-flop comprising...”, “A configurable MAC comprising...”, etc.).
- Clear reference to specific prior works (e.g., Razor FF, AMCAL, existing adaptive PM paper) and what you changed/improved.

5. Final presentation (15–20 minutes)

- Problem and motivation.
- Key idea and why it is new.
- Implementation flow and main results.
- Possible target IEEE venues (e.g., TVLSI, TCAS-I, JETCAS special issues).

You can then, after the course, refine the best project into a short IEEE conference paper.

2. Grade distribution (100%) – applies to all projects

1. Technical quality and correctness – 30%

- Correct functionality of the design (simulations, no major bugs).
- Proper use of Synopsys DC + ICC flow (synthesis + at least basic P&R, timing closure on a realistic constraint).
- Evidence of debugging and validation (testbenches, waveforms).

2. Novelty and analysis depth – 25%

- Clear explanation of what is new versus at least 2–3 recent papers.

- Meaningful quantitative comparison to a baseline (or more than one baseline).
- Sensitivity analysis: more than one operating point (e.g., voltage or constraint).

3. Documentation: report + patent page – 20%

- Well-structured, conference-style report with figures, tables, and references.
- Clear novelty/patentability page (bullets, diagrams).
- Correct writing, formatting, and citation style.

4. Implementation artifacts (code + scripts + reproducibility) – 15%

- Clean, modular RTL and commented scripts.
- README with steps to reproduce main results.
- Organized project directory and naming conventions.

5. Presentation and teamwork – 10%

- Clear oral presentation and slides.
- All members present and able to answer questions.
- Evidence of fair task distribution (each member's contribution visible).